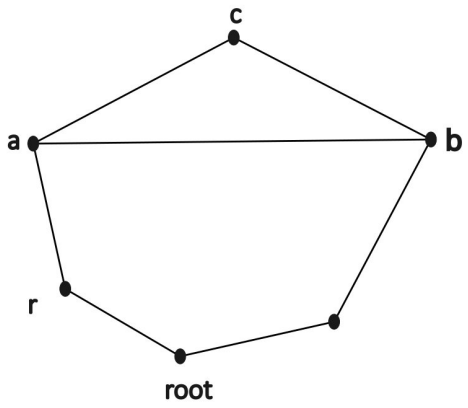
The algorithm is like a tree. We can find that in the structure.

Now to prove our algorithm we have to go sequentially to proof different element of the algorithm.

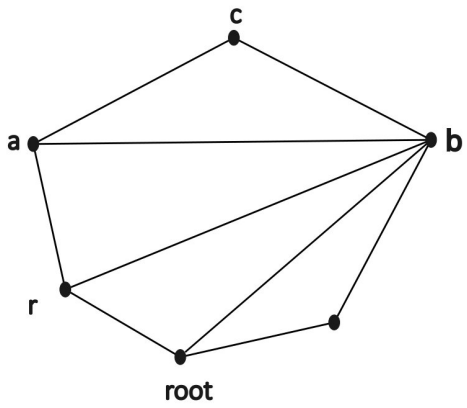
Firstly, we can go with the proof for completeness. Because as we know that there are prune operations for a particular triangulation, where a blocking chord of the triangulation is a duplicate chord. Here, we prune that triangulation and the branch because of that duplication. Now we have to prove that all of the children of that branch would have been invalid.

Now in the matter of invalid triangulations, we only notify those triangulations invalid, where aany one of the blocking chords are a duplocate chord, meaning the chord has occurred earlier faces. Now in the below section, we have to prove that a blocking chord is never flipped in the children of a branch.

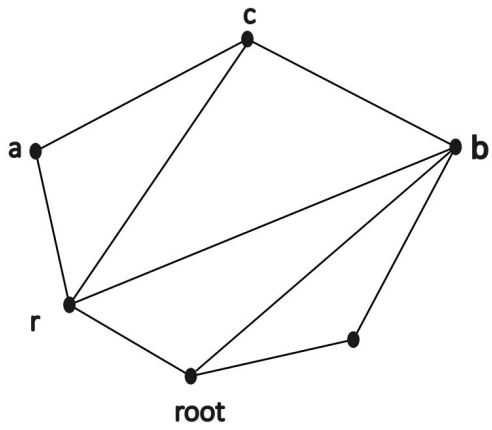
Let a chord be a-b which is a blocking chord in any triangulation tree. Now as the node is a triangulation, every face will be triangulated. Meaning there will be two other points who will make triangulations with a-b chord, in the two sides. We will select the point at the opposite side of the root vertex as c, and the point at the root side r. Now the r can be in different sections.



Now, firstly, the r could be on the left of the root vertex like this

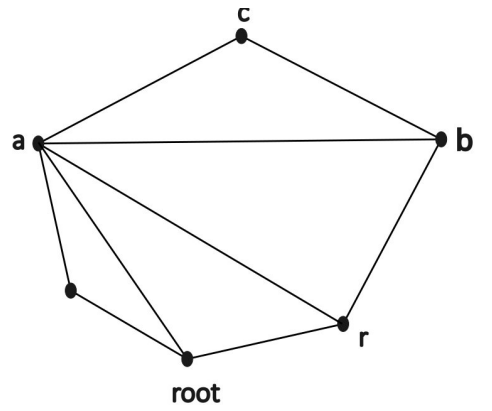


In this case, flipping the a-b chord will result in r-c chord.

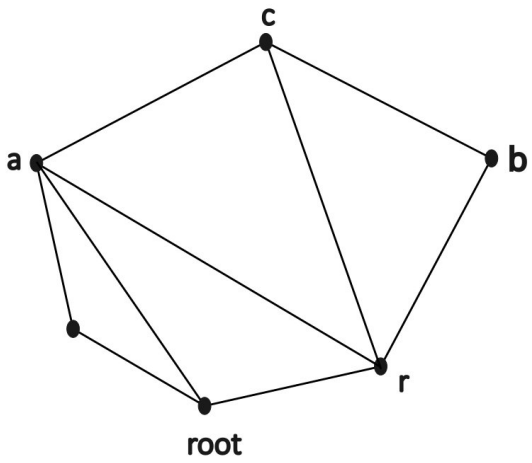


Notice that before the flipping the b-r chord were also a blocking chord, and it was left of a-b, meaning a-b could not be the left most blocking chord, as a result the new r-c also cant be the left most blocking chord, because you can see that the r-b chord is still there and still left side than the c-r node, meaning c-r do not have any chace for being the left most blocking chord of the triangulation. So, in this particular case, flipping a-b blocking chord is not possible according to the algorithm

Also if the r is on the right side of the root vertex

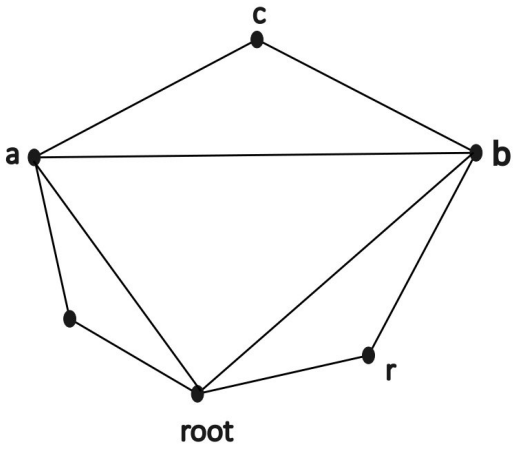


Here flipping the a-b chord will result in

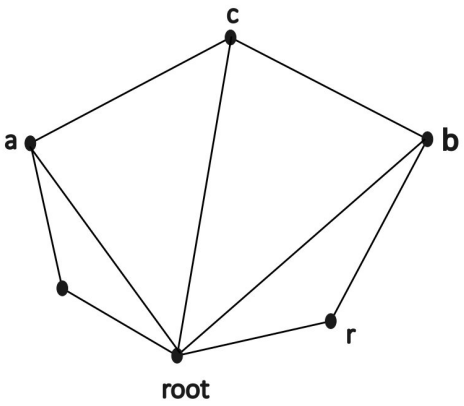


The new r-c chord. But notice that the a-r chord i more left than the r-c chord, so the flipping is not valid according to the algorithm as well.

Finally, if the r is at the root then



In this case flipping the a-b chord will result in



The new root-c chord is making the new child triangulation's root vertex with better visibility than the parent one, so the flipping is not valid as well.

So we can see in the all of those 3 cases, flipping a blocking chord in a tree of triangulation is impossible. Meaning that a blocking chord will be present in all of its children.

Thus, if a blocking chord is a duplicate chord, then we can say that all of the child triangulation will contain the chord which will make all of it child triangulations invalid.

So, by pruning them, we are not ignoring any valid triangulations. (Proved)

Now we have proved that in our pruning method of comparing blocking chords, we have not left out any valid triangulations.

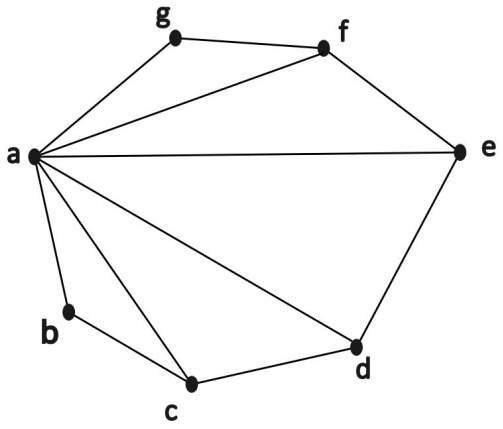
So once we can find a root triangulation, where no chord will be a duplicate or blocking chord, we can always find all of the valid triangulations via the algorithm. Now the question is can we always find such a valid root triangulation?

Proof: There will always be a valid root triangulation.

To elaborate this, we are calling a root triangulation a root of the tree of triangulations. Here we will select a root vertex and add chords from the root vertex to all of the non neighbouring chords of the root vertex. Thus we can get the root triangulation.

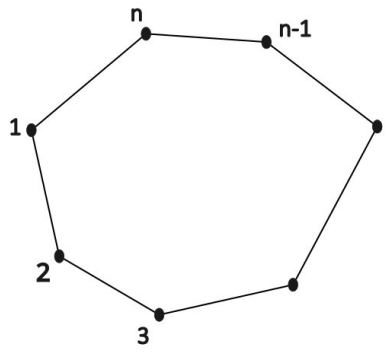
Now to have a root triangulation, we have find such a safe root, adding chords from it to every other non neighbouring nodes will not create multi edge problem.

Rephrasing that, we have to prove the existence of such a root, with whom none of the previous faces share a chord where one end is the root vertex and at the other end is one of the vertices of the current face.



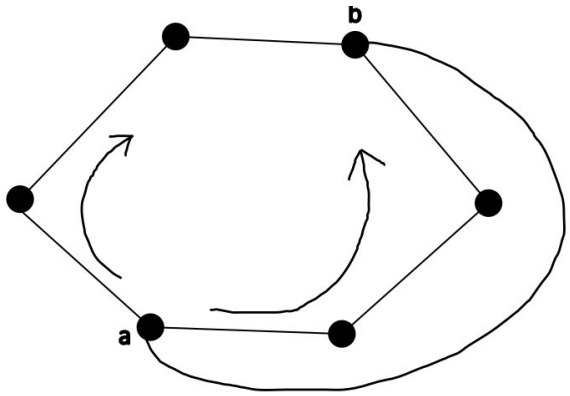
In this picture a vertex is the root vertex, adding chords with each of the non neighbouring nodes of a with 'a' vertex will get us the root triangulation of the tree of triangulations.

So we have to prove that in this face of abcdefg, there will always be safe vertices like a, creating root triangultions basing on them will not create any multi edge problem.

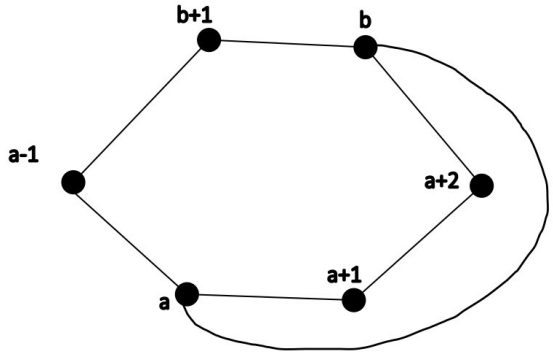


Now we got a face where there are n vertices, numbered from 1 to n. Now as it is a planar graph, there can be only chords in the outer plane. So there is one possiblity that there will not be any chords from the previous faces that will have both endpoints inside of the current face. In that particular case, all of the n vertices will be eligible for being a safe vertex.

But if there is atleast one chord on the outer plane, belonging to any of the previous faces, then

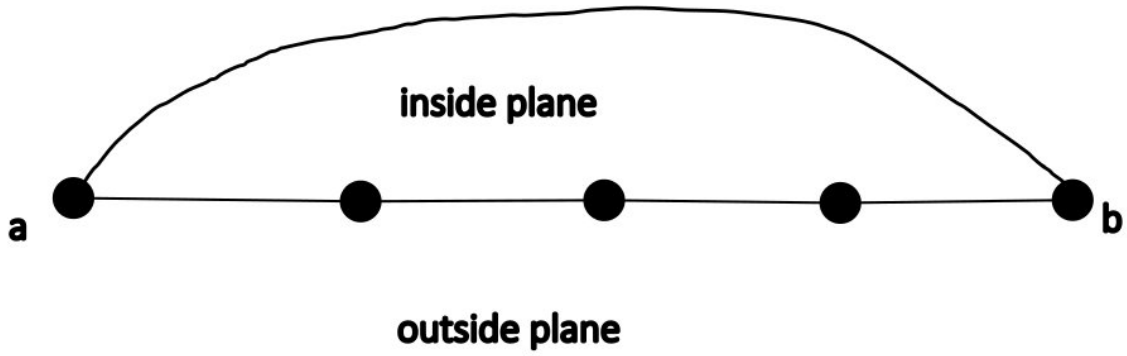


Here there is a chord on the outer plane, where a-b chord is present in one of the previous faces. Now a-b cannot be chord in the present face, meaning both a and b are disqualified from being a safe vertex in the current face. Now there is a very important contraint added in the current face.



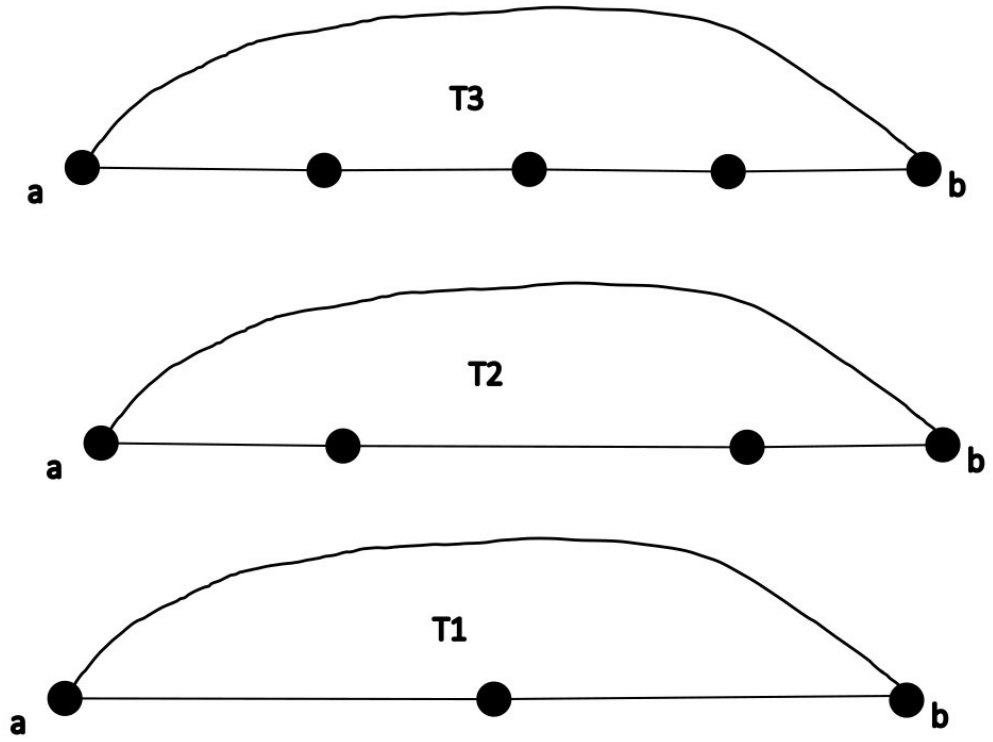
As our current graph is a planar graph and we are doing planar triangulation, any chord cannot overtake any other chords. Now as we have found a-b chord in here, that means there cannot be any chord on the outer plane that will overtake a-b chord. As a result, here in the picture, node (a+1, a+2) might have connection between them (if not neightbour) but they cant have a chord with any of the nodes (a-1, b+1), because that will break the planarity constraint by overtaking the a-b chord on the outer plane. So now to prove there will be any safe vertex, we have to just prove that between a+1 to b-1 there have to be a vertex that will not have any chord with a , a+ 1, ... ,b on the other hand, for a-1 ... B+1, we have to prove that there will be a vertex which will not have chords with a,a-1,.....,b+1,b .

Now we can easily find out that our connection has become like a specific structure like this.

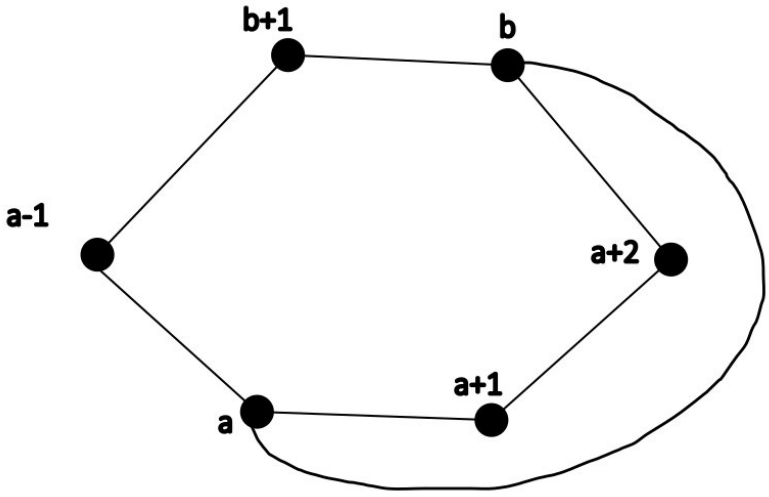


Here the internal vertices can make chords in the inside plane, but cant cross the vertex between start and end vertices, the a-b chord. Now there can be any number of vertices inside this particular structure. We can call them Tn structures, n is the number of vertices inside the a-

b vertices.



Now in this case,we have two of these Tn structures here in the face



One is the a,a+1,....,b and the other is a,a-1,....,b

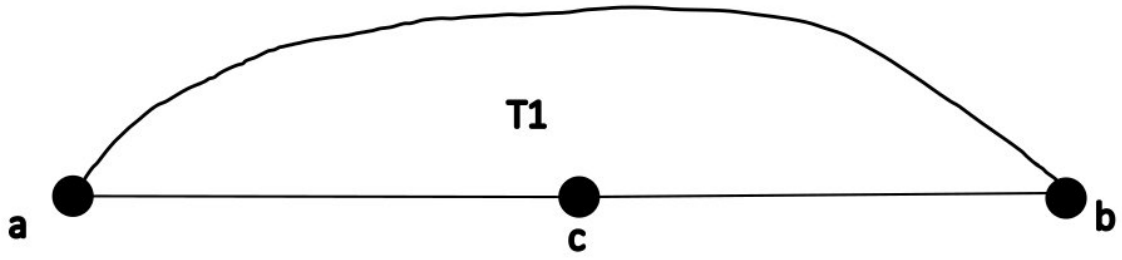
Now the two of the Tn structures has some internal vertices, 1,..,n-3

The lowest possible Tn is T1, there cannot be any T0, because that will mean a and b are neighbours, and no two neighbouring nodes can make a chord.

So the lowest is 1, and highest number is n - 1 - 2 = n - 3 (there 2 nodes minus are and b, and the one is T1 on the other end.)

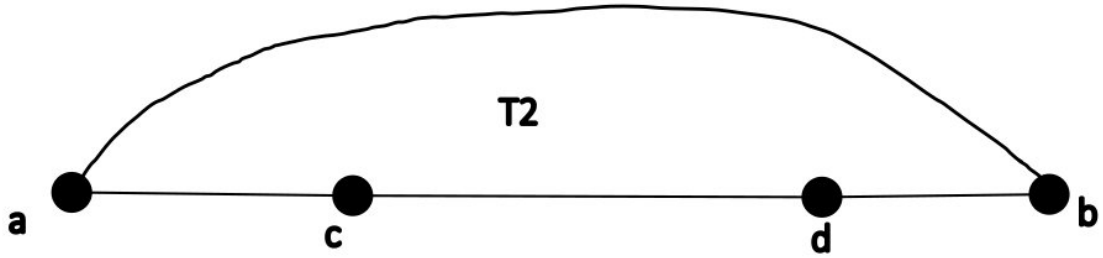
We can call the two Tn structures as the complenting Tn structures of one another.

Now for T1 we can proof that the one vertex inside it is the safe vertex, because

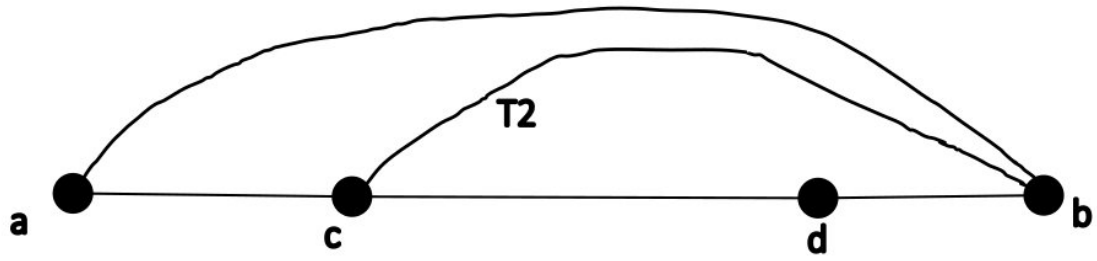


Here a and b both are the neighbors of c, that means any chord by c is impossible.

Now for T2,

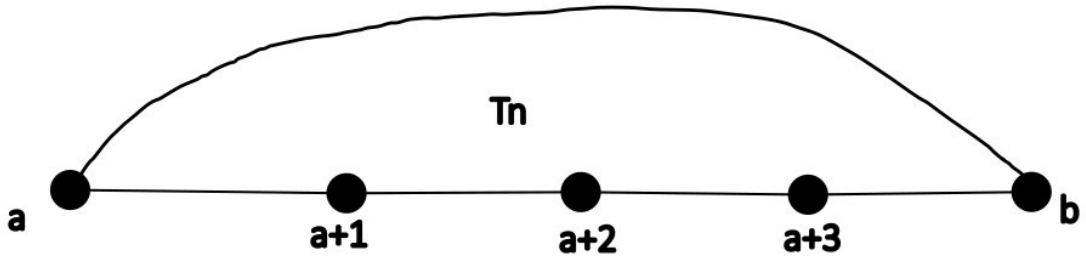


Here if there is a chord with c, it must be with b, because only b node is not neighbor with c, but this will turn the d vertex inside a T1 structure, making it a safe vertex.



The same will be true for d, as d can only build chords with a, which will automatically turn the c vertex inside a T1, rendering c as safe vertex.

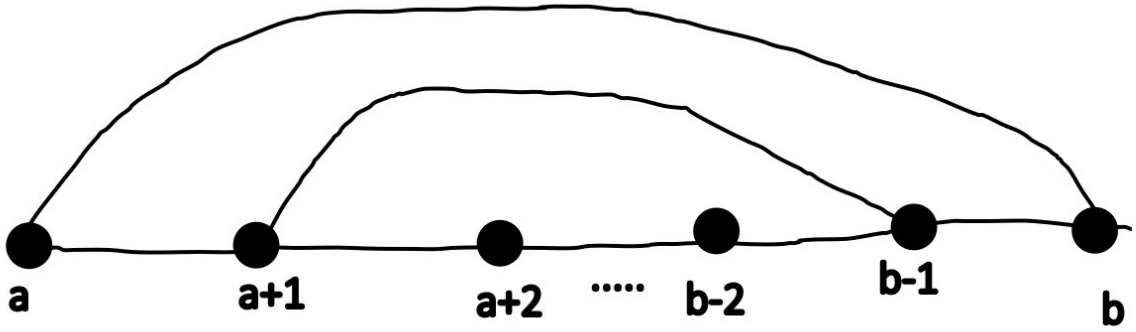
Now for a varible size Tn, lets check our algorithm to find the safe root



Here if we check a+1 vertex, it can not create chord with its neighbor a and a+2, meaning that its options are a+3,.....,b, now we start checking from b.

For each check, we are using a hashset, so the checking time complexity is O(1).

Now if the check returns true, meaning there is a a+1 and b chord, then the structure becomes a T(n-1) structure, where the a+2,...,b-1 vertices are now inside a new Tn structure. If the check returns false, then we retract and check b-1. In this process



Here every time based on the result of the check our n gets lower, because when a+1 failed to make a chord with b, then there can only be two things

1. A+1 has a chord with one or more from the set {b-1,b-2,....,a+3}
2. A+1 is a safe vertex, does not have a chord with anyone

Now for the first case, lets guess a+1 does have a chord with one of the lower rank than b, then in this case, T becomes T(n-2) (For a+1, b-1 chord)

So in the first case the n of Tn is always decreasing, so that will mean eventually it will come down to T1 or T2 which we have already proved does contain a safe vertex.

And for the second case, a+1 is itself a valid safe vertex, meaning coming from all down to checking b, b-1,b-2,.....,a+4,a+3, neither of them has any chord with a+1

So by this way, we can prove that we can get a safe vertex in O(n) time, also here also proves that every Tn structure must have at least one safe vertex.

As we know, from a face which have no multiedges in the previous faces, there can be all n number of safe vertices. But those faces that does have at least a multi edge in one of the previous faces, will must have at least two Tn structures, both of whom will must contain a safe vertex as we proved earlier.

This, every face in a biconnected planar graph, regardless of calculations stage, must have at least two safe vertices. (Proved) Thus there will always be at least two safe root triangulations regardless of the calculation order of the faces.

So we have both proved the completeness of our algorithm and the presence of safe root triangulation in any position of faces. Thus we can now say that our algorithm covers all of the valid planar triangulations of the planar biconnected graph..

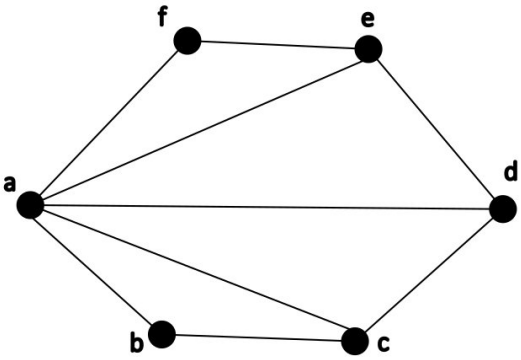
Time complexity proofs

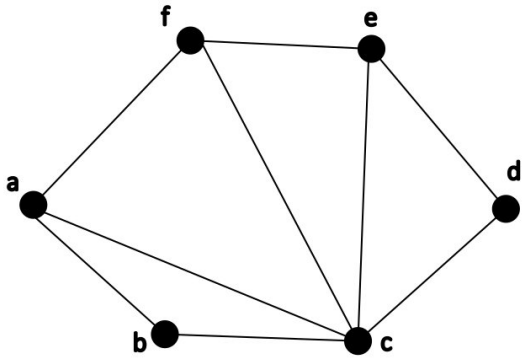
Now we move on into time complexity proofs. We have claimed earlier is that our algorithm is amortized O(1) or amortized linear. Now to proof this claim.

We need some other topics, like what is the least number of triangulations in a tree (with a initial building time)
we are calling the time building time, where we calculate the safe root , and the chords of the root vertex, fiom our algorithm and proves, the time for finding the safe vertex is O(n) and the time for adding all the chords is also O(n), meaning the time complexity for building a initial tree structure in any face is O(n),

Now we have already proved that our algorithm can find all valid triangulations in a tree structures by flipping. But the question is , how many of the valid triangulations are actually are there. I mean if there is only one valid triangulation for a building time of n=1000, then our algorithm is clearly not linear.

So, from the proofs, we have already found out that there is at least two safe vertices in a biconnected planar graphs triangulation, in any order of calculation of the faces.





So by the proof of our algorithm, if we pick one of them as our root triangulation and then continue, we should find the other combinations, like starting with a root will must find the triangulation which have c root, and vice versa.

Now in one stage we can only flip one chord. By also flipping the one one chord we will find a triangulation that is also a valid triangulation(because if it was not, it would have been pruned and we would fail to reach the c root triangulation.)

Now we see how many chords does the two a root and c root shares.

For a root triangulation all chords has one endpoint a and the other variable

For c root triangulation all chords has one endpoint c and the other variable

So in this case only one chord might be same a-c, but all other are different

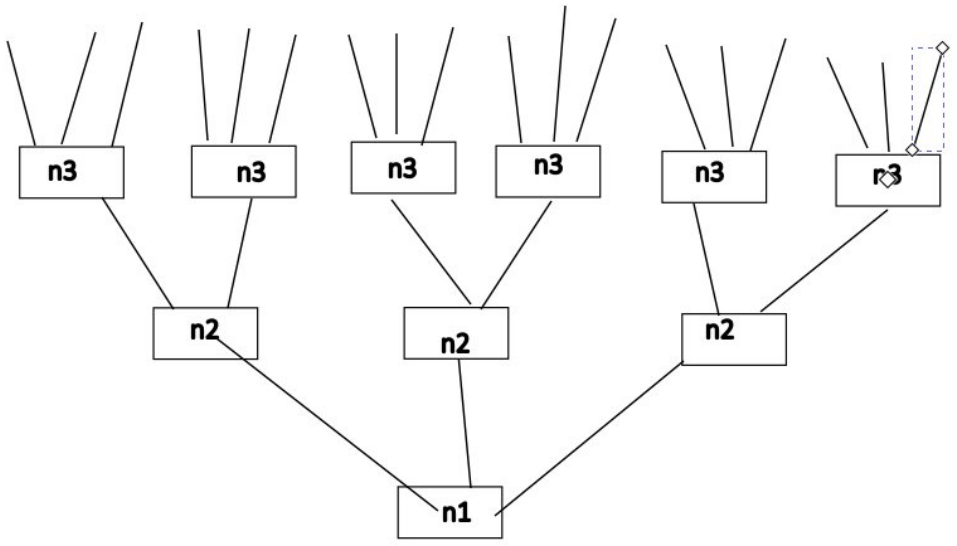
Meaning all of the n-4 chords must be changed to react from a root to c root triangulation.

That means n - 4 flipping, each of them wil be a valid triangulation.

That means, including the root triangulation, there will at least be n - 4 + 1 = n - 3 triangulations in the face.

So there will at least be O(n) triangulations in a face with n vertices.

Now as we know our structure is like a recursive tree structure.



Here we have found that the n1,n2,n3 is the number of vertices of each faces, here kind of as levels. Now we already know that in a node in the above tree , we have to do O(n) work for building the tree structures of the tree of triangulations and also to find the safe vertex. So in this case, we have to find how many time this building time actually affects the final time.

Now for face 1, we just have to build the tree once and we can get n2 trees by flipping, so total time for going to n2, is

N1 + n2 * n1 is the total set up time for getting n1*n2 triangulations for a 2 face biconnected graph.

= n1 + n1 * n2

Now we know n2 >= 3

So n1 * 1 < n1 * n2

So total building time n1 + n1 * n2

= n1 * 1 < n1 * n2

= n1 * 1 + n1 * n2 < n1 * n2 * 2

= total buidling time < total triangulations * 2

= B < T * 2

So total time time complexity is less that (total triangulations * 3)

So its amortized linear

Now for a 3 face graph

The total set up time is n1 + n1 * n2 + n1 * n2 * n3 and the total triangulations are n1 * n2 * n3

Now n1 + n1 * n2 + n1 * n2 * n3 = B (total building time)

Also n1 * n2 * n3 = T (total triangulation time)

As we know n1 + n1 * n2 < n1 * n2 * 2

= B < n1 * n2 * 2 + n1 * n2 * n3

But we also know that

3 <= n3

So 2 < n3

= n1 * n2 * 2 < n1 * n2 * n3

= n1 * n2 * 2 + n1 * n2 * n3 < n1 * n2 * n3 * 2

So

= B < n1 * n2 * n3 * 2

= B < T * 2

Total triangulations time T < n1 * n2 * n3 * 3

Still total time time complexity is less that (total triangulations * 3)

Now we can extend it to any value n_i as we want

Still for 4 faces

B = n1 + n1 * n2 + n1 * n2 * n3 + n1 * n2 * n3 * n4

T = n1 * n2 * n3 * n4

Here we already know

= n1 + n1 * n2 + n1 * n2 * n3 < n1 * n2 * n3 * 2

So

B < n1 * n2 * n3 * 2 + n1 * n2 * n3 * n4

But we also go like the previous time 2 < n4 (n4 >= 3)

So n1 * n2 * n3 * 2 < n1 * n2 * n3 * n4

= B < n1 * n2 * n3 * n4 * 2

= B < T * 2

So total triangulation time < triangulation count * 3

It is true for any number of faces which has vertex number >= 3

That means for any biconnected planar graph ,

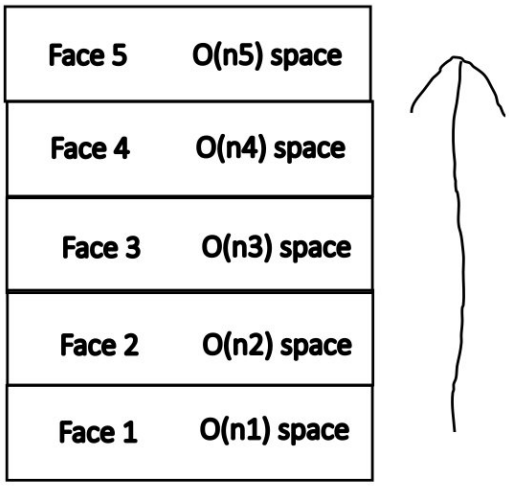
this algorithm takes total time < triangulations * 3 time complexity

So this algorithm is amortized linear(Proved)

Now the space complexity

We already know from the original algorithm , that the space complexity is O(n)

Here for a particular face



Here when we move to face i to face i+1, we have to save all information for faces 1..i n1+...+ni

Now as we know here ni is the number of vertices, and we also know for a planar graph the total chords complexity is also O(n), as n-3 chords will be there for a particular time. So n1+..+ni where i is the total face number will be O(n) where n is the total number of vertices.

Also we need a hashset as well for storing the current chord information. So, we will keep a hashset as well, where it will also be O(n) space complexity.

As a result our whole algorithm is taking O(n) space compexity and amortized O(1) time complexity (Proved)